

The New Mirror

VOL. 1, NO. 011

2026-03-30

COVER STORY

Job negotiation for programmers: the basic principles

You need to negotiate at a new job: for your salary, or benefits, or my personal favorite, a shorter workweek.

Itamar Turner-Trauring

You're not sure what to do, or how to approach it, or what to say when the company says "how much do you want?" or "here's our offer—what do you say?"

Here's the thing: that final conversation about salary might be the most nerve-wracking part, but the negotiation process starts much much earlier. Which means you can enter that final conversation having positioned yourself for success—and feeling less stressed about it too.

The way you can do that is following certain basic principles, which I'll be covering in this article. I'm going to be focusing on salary negotiation as an example, but the same principles will apply when negotiating for a shorter workweek.

In particular, I'll be talking about:

An example from early in my career when I negotiated very very badly.

The right way to negotiate, based on four principles:

Employment is a negotiated relationship.

Knowledge is power.

Negotiate from a position of strength.

Use the right tactics.

id="the-wrong-way-to-negotiate">The wrong way to negotiate

Before moving on to the principles of negotiation, let me share a story of how I negotiated badly.

During my first real job search I interviewed at a company in New York City that was building a financial trading platform. They were pretty excited about some specific technologies I'd learned while working on Twisted, an open source networking framework. They offered me a job, I accepted, and my job search was over.

But then they sent me their intellectual property agreement, and I actually read legal documents; you should read them too. The agreement would have given the company ownership over any open source work I did, including work on Twisted. I wanted to ensure I could keep doing open source development, especially given that was their reason for hiring me in the first place. I asked for an exemption covering Twisted, they wouldn't agree, and so we went back and forth trying to reach an agreement.

Eventually they came back with a new offer: in return for not working on Twisted I'd get a 20% salary increase over their initial offer. I thought about it briefly, then said no and walked away from the job. Since I had neither a CS degree—I'd dropped out—nor much of an employment history, open source contribution was important to my career. It was how I'd gotten contracting work, and it was the reason they'd offered me this job. And I enjoyed doing it, too, so I wasn't willing to give it up.

I posted about this experience online, and an employee of ITA Software, which was based in the Boston area, suggested they were happy to support contributions to open source projects. It seemed worth a try, so I applied for the position. And when eventually I got a job offer from ITA and they asked me for my salary requirements, I asked for the

second offer I'd gotten, the one that was 20% higher than my original offer. They accepted, and I've lived in the Boston area ever since.

As we go through the principles below, I'll come back to this story and point out how they were (mis)applied in my two negotiations.

id="the-four-principles-of-negotiation">The four principles of negotiation

You can think of the negotiation process as building on four principles:

Employment is a negotiated relationship.

Knowledge is power.

Negotiate from a position of strength.

Use the right tactics.

Let's go through them one by one.

id="principle-1-employment-is-a-negotiated-relationship">Principle #1: Employment is a negotiated relationship

If you're an employee, your employment relationship was negotiated. When you got a job offer and accepted it, that was a negotiation, even if you didn't push back at all. Your choice isn't between negotiating and not negotiating: it's between negotiating badly, or negotiating well.

id="negotiate-actively">Negotiate actively

If you don't actively try to negotiate, if you don't ask for what you want, if you don't ask for what you're worth — you're unlikely to get it. Salaries, for example, are a place where your interests and your employer's are very much at odds. All things being equal, if you're doing the exact same work and have the same likelihood of leaving, would your employer prefer to pay you less or more? Most employers will pay you less if they can, and I almost had to learn that the hard way.

Applying the principle: In my story above, I never proactively negotiated. Instead, I accepted a job offer from the financial company without any sort of additional demands. If they were happy to offer me a 20% raise just to quit open source, I probably could have gotten an even higher salary if I'd just asked in the first place.

id="negotiation-starts-early-and-never-ends">Negotiation starts early, and never ends

Not only do you need to negotiate actively, you also need to realize that negotiation starts much earlier than you think, and ends only when you leave to a different job:

The minute you start thinking about applying to a company, you've started the negotiation process; as you'll see, you'll want to do research before you even talk to them.

Your interview is part of your negotiation, and you can in fact negotiate the interview process itself (e.g. suggest sharing a code sample instead of doing a whiteboard puzzle).

As an employee you will continue to negotiate: if you always say "yes" when your boss asks you to work long hours, your contract for a 3-day weekend will mean nothing.

In short, your whole relationship as an employee is based on negotiation.

id="distinguish-between-friend-and-foe">Distinguish between friend and foe

A negotiation involves two sides: yours, and the company's. When you're negotiating it's important to remember that anyone who works for the company is on the company's side. Not yours.

I once had to negotiate the intellectual property agreement at a new job. My new employer was based in the UK, and it had a US subsidiary organized by a specialist company. These subsidiary specialists had provided the contract I was signing.

When I explained the changes I wanted to make, the manager at the subsidiary specialist told me that my complaint had no merit, because the contract had been written by the "best lawyers in Silicon Valley." But the contract had been written by lawyers working for the company, not for me. If his claim had been true (spoiler: they were not in fact the best lawyers in Silicon Valley), that would have just made my argument stronger. The better the company's lawyers, the more carefully I ought to have read the contract, and the more I ought to have pushed back.

The contracts the company wants you to sign? They were written by lawyers working for the company.

Human Resources works for the company, as does the in-house recruiter. However friendly they may seem, they are not working for you. And third-party recruiters are paid by the company, not you. It's true that sometimes their commission is tied to your salary, which means they would rather you get paid more. But since they get paid only once per candidate, volume is more important than individual transactions: it's in their best interest to get you hired as quickly as possible so they can move on to placing the next candidate.

Since all these people aren't working for you, during a negotiation they're working against you.

The only potential exception to this rule are friends who also work for the company, and aren't directly involved in the negotiation process: even if they are constrained in some ways, they're probably still on your side. They can

serve as a backchannel for feedback and other information that the company can't or won't share.

id="principle-2-knowledge-is-power">Principle #2: Knowledge is power

The more you know about the situation, the better you'll do as a negotiator. More knowledge gives more power: to you, but also to the company.

id="know-what-you-want">Know what you want

The contracts the company wants you to sign? They were written by lawyers working for the company.

Itamar Turner-Trauring

The first thing you need to do when negotiating is understand what you want.

What is your ideal outcome?

What can you compromise on, and what can't you compromise on?

What is the worst outcome you're willing to accept?

id="do-your-research">Do your research

You also want to understand where the other side is coming from:

What is the company's goal, and the negotiator's goal? For example, if you discover their goal is minimizing hassle, you might be able to get what you want by making the process a little smoother.

What resources are available to them? An unfunded startup has different resources than a large company, for example.

Has the company done something similar in the past, or will your request be unprecedented? For example, what hours do other employees in similar positions work? How much are other employees paid?

What do other companies in the area or industry provide?

How is this particular business segment doing: are they losing money, or doing great?

The more you understand going in, the better you'll do, and that means doing your research before negotiation starts.

Applying the principle: In my story above I never did any research about salaries, either in NY or in Boston. As a result, I had no idea I was being offered a salary far below market rates.

As a comparison, here's a real example of how research can help your negotiation, from an engineer named Adam:

Adam: "Being informed on salaries really helped my negotiating position. When my latest employer made me an offer I asked them why it was lower than their average salary on Glassdoor.com. The real reason was likely 'we offer as little as possible to get you on board.' They couldn't come up with a convincing reason and so the salary was boosted 10%."

Glassdoor is a site that allows employees to anonymously share salaries and job reviews. Five minutes of research got Adam a 10% raise: not bad at all!

id="listen-and-empathize">Listen and empathize

If you only had to make yourself happy this wouldn't be a negotiation: you need to understand the other side's needs and wants, what they're worrying about, what they're feeling. That means you need to listen, not just talk: if you do, you will often gather useful information that can help you make yourself more valuable, or address a particular worry. And you need to feel empathy towards the person you're talking to: you don't need to agree or subordinate yourself to their goals, but you do need to understand how they're feeling.

id="share-information-carefully">Share information carefully

Sharing information at the wrong time during a negotiation can significantly weaken your position. For example, sharing your previous salary will often anchor what the company is willing to offer you:

Adam: "I graduated from university and started working at the end of 2012. At my first job I worked for way under my market rate. I knew this and was OK with it because they were a good company.

Then I switched jobs in 2013. What I hadn't accounted for was that my salary at my first job was going to limit my future salary prospects. I had to fight hard for raises at my next job before I was in line with people straight out of school, because they didn't want to double my salary at my previous company."

In general, when interviewing for a job you shouldn't share your previous salary, or your specific salary demands—except of course when it is helpful to do so. For example, let's say you're moving from Google to a tiny bootstrapped startup, and you know you won't be able to get the same level of salary. Sharing your current salary can help push your offer higher, or used as leverage to get shorter hours: "I know you can't offer me my previous salary of \$\$\$, but here's something you could do—". Just make sure not to share it too early, or they might decide you'd never accept any offer at all and stop the interview process too early.

Most of the time, however, you shouldn't share either your previous salary or specific salary requirements. If the company insists on getting your previous salary, you can:

If you work somewhere with relevant laws (e.g. California and Massachusetts), point out that this question is illegal. Asking about salary expectations is not illegal in these jurisdictions, so be careful about the distinction.

Ask for the company's salary range for the position, as well as the next level up in the salary tree. Chances are they will refuse to share, in which case you can correspondingly refuse to share your information.

Say something like "I expect to be paid industry-standard pay for my experience."

Applying the principle: I shouldn't have told ITA Software my salary requirement. Instead, I should have gotten them to make the first offer, which would have given me more information about what they were willing to pay.

id="principle-3-negotiate-from-a-position-of-strength">Principle #3: Negotiate from a position of strength

The stronger your negotiation position, the more likely you are to get what you want. And this is especially important when you're asking for something abnormal, like a 3-day weekend.

id="have-a-good-fallback-batna">Have a good fallback (BATNA)

If negotiation fails, what will you do? Whatever it is, that is your fallback, sometimes known as the "Best Alternative to a Negotiated Agreement" (BATNA). The better your fallback, the better your alternative, the stronger your negotiating position is. Always figure out your fallback in advance, before you start negotiating.

For example, imagine you're applying for a new job:

If you're unemployed and have an empty bank account, your fallback might be moving in with your parents. This does not give you a strong negotiating position.

If you're employed, and more or less content with your current job, your fallback is staying where you are. That makes your position much stronger.

If you have a strong fallback, you can choose to walk away at any time, and this will make asking for more much easier.

id="provide-and-demonstrate-value">Provide and demonstrate value

The more an organization wants you as an employee, the more they'll be willing to offer you. The people you're negotiating with don't necessarily know your value: you need to make sure they understand why you're worth what you're asking.

For example, when you're interviewing for a job, you need to use at least part of the interview to explain your value

to your prospective employer: your accomplishments and skills. Once you've established the value of your skills, asking for more—more money, unusual terms—can actually make you seem more valuable. And having another job offer—or an existing job—can also help, by showing you are in demand.

Finally, remember that your goal is to make sure the other side's needs are met—not at your own expense, but if they don't think hiring you is worth it, you aren't going to get anything. Here's how Alex, another programmer I talked to, explains how he learned this:

Alex: "Think about the other person and how they're going to react, how you can try to manage that proactively. You need to treat your negotiating partner as a person, not a program.

Initially I had been approaching it adversarially, 'I need to extract value from you, I have to wrestle you for it' but it's more productive to negotiate with an attitude of 'we both need to get our needs met.' The person you're talking to is looking to hire someone productive who can create value, so figure out how can you couch what you want in a way that proactively addresses the other person's concerns."

id="principle-4-use-the-right-tactics">Principle #4: Use the right tactics

Once you've realized you're negotiating, have done your research, and are negotiating from a position of strength, applying the right negotiation tactics will increase your chances of success even more.

id="ask-for-more-than-you-want">Ask for more than you want

Obviously you don't want to ask for less than what you want. But why not ask for exactly what you want?

First, it might turn out that the company is willing to give you far more than you expected or thought possible.

Second, if you ask for exactly what you want there's no way for you to compromise without getting less than what you want. By asking for more, you can compromise while still getting what you wanted.

Applying the principle: If I'd wanted a \$72,000 salary, and research suggested that was a fair salary, I should have asked for \$80,000. If I was lucky the company would have said yes; if they wanted to negotiate me down, I would have no problems agreeing to a lower number so long as it was above \$72,000.

id="negotiate-multiple-things-at-once">Negotiate multiple things at once

Your goal when negotiating is not to "win." Rather, your goal is to reach an agreement that passes your minimal bar, and gets you as much as is feasible. Feasibility means you also

need to take into account what the other side wants as well. If you've reached an impasse, and you still think you can make a deal that you like, try to come up with creative ways to work out a solution that they will like.

If you only negotiate one thing at once, every negotiation has a winner and a loser. For example, if all you're negotiating is salary, either you're making more money, or the company is saving money: it's a zero-sum negotiation. This limits your ability to come up with a solution that maximizes value for you while still meeting the other side's needs.

Applying the principle: In my story above, the financial company wanted intellectual property protection, I wanted to be able to write open source, and we were at an impasse. So they expanded the scope of the negotiation to include my salary, which allowed them to make tradeoffs between the two—more money for me in return for what they wanted. If I'd cared less about working on open source I might have accepted that offer.

id="never-give-an-answer-immediately">Never give an answer immediately

During the actual negotiation you should never decide on the spot, nor are you required to. If you get a job offer you can explain that you need a little time to think about it: say something like "I have to run this by my spouse/significant other/resident expert." This will give you the time to consider your options in a calmer state of mind, and not just blurt out "yes" at the first semi-decent offer.

Having someone else review the offer is a good idea in general; a friend of mine ran her job offers by her sister, who had an MBA. But it's also useful to mention that other person as someone who has to sign off on the offer. That gives you the ability to say you'd like to accept an offer, but your spouse/expert thinks you can do better.

Notice that the employer almost always has this benefit already. Unless you're negotiating with the owner of the business, you're negotiating with an agent: someone in HR, say. When you make a demand, the HR person might say "I have to go to check with the hiring manager", and when they come back with less than you wanted it's not their fault, they're just passing on the bad news. The implication is that the low offer is just the way it is, and there's nothing they can do about.

Don't fall for this trick: they often can change the offer.

id="beyond-negotiating-for-salary">Beyond negotiating for salary

You can negotiate for a higher salary—or rather, you should negotiate for a higher salary. The Adam I interviewed in this article is now a partner in DangoorMendel, who can help you negotiate a higher salary .

But salary isn't the only thing you can negotiate for. You can also negotiate for a shorter workweek.

And yes, this is harder, but it's definitely possible.

In fact, this article is an excerpt from a book I wrote to help you do just that: [You Can Negotiate a 3-Day Weekend](#) .

Tired of scrambling to get your job done?

If you were productive enough, you could take the afternoon off, confident you'd produced high value work. Not to mention having an easier time finding a new job when you need one.

[Learn the secret skills of productive programmers](#) .

FRONT PAGE

ADL Bias and Diversity Training

Scott Johnson (fuzzyblog)

As a contractor for the ADL, I recently went through our official bias and diversity training. I put together my notes on that for a new hire who recently joined us.

Here are some of the discussion questions:

Is it ok for people of a group to use slurs about their own identity?

Should airlines be required to provide multiple seats to obese people ?

Does your work place treat everyone fairly?

Can anyone be successful in the USA?

Does the USA live up to its ideals of racial diversity?

Do women have the same opportunities as men?

Do people of color have the same opportunities as white people?

Peanut butter racism

Blind spot bias you may have bias that you see in others but can't see in yourself

White fragility by d Angelo

MTV micro aggressions

(my personal favorite (this is a Scott thing)- Tea and Consent)

}

{

FEATURES

On taking good care of your phone

Drew DeVault

I just finished replacing the micro-USB daughterboard on my Samsung Galaxy S5, which involved taking the phone most of the way apart, doing the replacement, and putting it back together. This inspired me to write about my approach to maintaining my cell phone. I've had this phone for a while and I have no plans to upgrade - I backed the upcoming Purism phone, but I expect to spend months/years on the software before I'll be using that as my daily driver.

I don't want to be buying a new phone every year. That's a lot of money! Though the technophile in me finds the latest and greatest technology appealing, the thought of doing my own repairs and upkeep on a battle-tested phone is equally interesting. Here are the four things I've found most important in phone upkeep.

id="install-lineageos-or-replicant">Install LineageOS or Replicant

Before I installed CyanogenMod when I bought this phone, I did some prying into the stock ROM to see just how bad it was. It was even worse than I expected! There were literally hundreds of apps and services with scary permissions running in the background that could not be removed. These spy on you, wear down your battery, and slow down your phone over time - another form of planned obsolescence.

My phone is still as fast as the day I got it. It does a great job with everything I ask it to do. The first thing you should do with every new phone is install a third-party ROM - ideally, without Google apps. Stock ROMs suck, get rid of it.

id="insist-on-a-user-replacable-battery">Insist on a user-replacable battery

Non-user-replacable batteries are an obvious form of planned obsolescence. Batteries don't last forever and you should never buy a phone that you cannot replace the battery of. A new battery for my S5 costs 10 bucks. 4 years in, I've replaced mine once and I can hold a charge fine for a couple of days.

id="get-a-case">Get a case

This one is pretty obvious, but I didn't follow this advice at first. I've never broken a screen, so I didn't bother with a case. When I decided I was going to keep this phone for a long time, I went ahead and bought one. It doubles the thickness of my phone but at least I can be sure I'm not going to bust it up when I drop it. It still fits in my pocket comfortably so it's no big deal.

id="attempt-repairs-before-you-buy-a-new-phone">Attempt repairs before you buy a new phone

The past couple of months, my phone's micro-USB3 port started to act up a bit. I would have to wiggle the cable a bit to get it to take, and it could stop charging if I rustled

my desk the wrong way. I got a replacement USB daughterboard on Amazon for 6 bucks. Replacing it took an hour, but when removing the screen I broke the connection between my home button and my motherboard - which was only 10 bucks for the replacement, including same day shipping. The whole process was a lot easier than I thought it would be.

Be a smart consumer when you're buying a phone. Insist on the replacable battery and maybe read the iFixit teardown. Take good care of it and it'll last a long time. Don't let consumerism get the better of you!

}

{

Drew DeVault

Gemini is a network protocol for exchanging hypertext documents — “hypertext” in the general sense of the word, not with respect to the hypertext markup language (HTML) that web browsers understand. It’s a simple network protocol which allows clients to request hypertext documents (in its own document format, gemtext). It is, in some respects, an evolution of Gopher , but more modernized and streamlined.

```
$ openssl s_client -quiet -crlf \ -servername
drewdevault.com \ -connect drewdevault.com:1965 \ | awk
'{ print "response: " $0 }' gemini://drewdevault.com re-
sponse: 20 text/gemini response: ``ASCII art of a rocket
next to "Drew DeVault" in a stylized font response: ^
response: || _____ _ . _ _ response:
||\_____\_____--\_\\\__/_|
|_|_|_ response: /|||\_ _ \_ \|V V /||\ _ Y^ _ \|
\\ _ \ response: :|:|\ ` \| \ _ ^ / | ` _ ^ // _ \| | | |
response: |:|:|/_/_/_ > \_// _ _ ^ _ > \_/
( _/_/_/_/_/_ response: /||| V V V V V response: **
response: ** response: `` [...]
```

My disdain for web browsers is well documented ¹. Web browsers are extraordinarily complex, and any attempt to build a new one would be a Sisyphean task. Successfully completing that implementation, if even possible, would necessarily produce a Lovecraftian mess: unmaintainable, full of security vulnerabilities, with gigabytes in RAM use and hours in compile times. And given that all of the contemporary web browsers that implement a sufficiently useful subset of web standards are ass and getting assier, what should we do?

something different, like Gemini. Gemini does not solve all of the web's problems, but it addresses a subset of its use-cases better than the web does, and that excites me. I want to discard the parts of the web that Gemini does better, and explore other solutions for anything that's left of the web which is worth keeping (hint: much of it is not).

It's dead simple. A client or server implementation can be written from scratch by a single person in the space of an afternoon or two. A new web browser could take hundreds of engineers millions of hours to complete.

It's opinionated about document formatting. There are no inline links (every link goes on its own line), no formatting, and no inline images. Gemini strictly separates the responsibility of content and presentation. Providing the content is the exclusive role of the server, and providing the presentation is the exclusive role of the client. There are no stylesheets and authors have very little say in how their content is presented. It's still possible for authors to express themselves within these constraints — as with any other constraints — but it allows clients to be simpler and act more as user agents than vendor agents.

For my part, I have been working on a number of Gemini projects. For one, this blog is now available on Gemini , and I have started writing some Gemini-exclusive content for it. I've also written some software you're welcome to use:

gmnsrv is a high-performance Gemini server, also written in C11 for POSIX systems with OpenSSL. It supports zero-configuration TLS, CGI scripting, auto-indexing, regex

}

{

Rust for Linux revisited

Drew DeVault

Ugh. Drew's blogging about Rust again.

– You

I promise to be nice.

Two years ago, seeing the Rust-for-Linux project starting to get the ball rolling, I wrote “Does Rust belong in the Linux kernel?”, penning a conclusion consistent with Betteridge's law of headlines. Two years on we have a lot of experience to draw on to see how Rust-for-Linux is actually playing out, and I'd like to renew my thoughts with some hindsight – and more compassion. If you're one of the Rust-for-Linux participants burned out or burning out on this project, I want to help. Burnout sucks – I've been there.

The people working on Rust-for-Linux are incredibly smart, talented, and passionate developers who have their eyes set on a goal and are tirelessly working towards it – and, as time has shown, with a great deal of patience. Though I've developed a mostly-well-earned reputation for being a fierce critic of Rust, I do believe it has its place and I have a lot of respect for the work these folks are doing. These developers are ambitious and motivated to make an impact, and Linux is undoubtedly the highest-impact software in the world, and in theory Linux is enthusiastically ready to accept motivated innovators into its fold to facilitate that impact.

At least in theory. In practice, the Linux community is the wild wild west, and sweeping changes are infamously difficult to achieve consensus on, and this is by far the broadest sweeping change ever proposed for the project. Every subsystem is a private fiefdom, subject to the whims of each one of Linux's 1,700+ maintainers, almost all of whom have a dog in this race. It's herding cats: introducing Rust effectively is one part coding work and ninety-nine parts political work – and it's a lot of coding work. Every subsystem has its own unique culture and its own strongly held beliefs and values.

The consequences of these factors is that Rust-for-Linux has become a burnout

machine. My heart goes out to the developers who have been burned in this

project. It's not fair. Free software is about putting in the work, it's a

classical do-ocracy... until it isn't, and people get hurt. In spite of my

critiques of the project, I recognize the talent and humanity of everyone

involved, and wouldn't have wished these outcomes on them. I also have sympathy

for many of the established Linux developers who didn't exactly want this on

their plate... but that's neither here nor there for the purpose of this post,

and any of those developers and their fiefdoms who went out of their way to make

life difficult for the Rust developers above and beyond what was needed to

ensure technical excellence are accountable for these shitty outcomes. 1

So where do we go now?

Well, let me begin by re-iterating something from my last article on the subject: “I wish [Rust-for-Linux] the best of luck and hope to see them succeed”. Their path is theirs to choose, and though I might advise a moment to rest before diving headfirst into this political maelstrom once again, I support you in your endeavours if this is what you choose to do. Not my business. That said, allow me to humbly propose a different path for your consideration.

Here's the pitch: a motivated group of talented Rust OS developers could build a Linux-compatible kernel, from scratch, very quickly, with no need to engage in LKML politics. You would be astonished by how quickly you can make meaningful gains in this kind of environment; I think if the amount of effort being put into Rust-for-Linux were applied to a new Linux-compatible OS we could have something production ready for some use-cases within a few years.

Novel OS design is hard: projects like Redox are working on this, but it will take a long time to bear fruit and research operating systems often have to go back to the drawing board and make major revisions over and over again before something useful and robust emerges. This is important work – and near to my heart – but it's not for everyone. However, making an OS which is based on a proven design like Linux is much easier and can be done very quickly. I worked on my own novel OS design for a couple of years and it's still stuck in design hell and badly in need of being rethought; on the other hand I wrote a passable Unix clone alone in less than 30 days.

Rust is a great fit for a large monolithic kernel design like Linux. Imagine

having the opportunity to implement something like the dcache from scratch in

Rust, without engaging with the politics – that's something a small group of

people, perhaps as few as one, could make substantial inroads on in a short

period of time taking full advantage of what Rust has on offer. Working towards

}

{

You don't need jQuery

Drew DeVault

It's true. You really don't need jQuery. Modern web browsers can do most of what you want from jQuery, without jQuery.

For example, take MediaCrush . It's a website I spent some time working on with a friend. It's actually quite sophisticated - drag-and-drop uploading, uploading via a hidden form, events wired up to links and dynamically generated content, and ajax requests/file uploads, the whole she-bang. It does all of that without jQuery. It's open source , if you're looking for a good example of how all of this can be used in the wild.

Let's walk through some of the things you like jQuery for, and I'll show you how to do it without.

id="document-querying-with-css-selectors">Document Querying with CSS Selectors

You like jQuery for selecting content. I don't blame you - it's really cool. Here's some code using jQuery:

Now, here's how you can do it on vanilla JS:

You want to make requests in JavaScript. This is how you POST with jQuery:

Here's the same code, without jQuery:

```
class="highlight"> var xhr = new XMLHttpRequest (); /\nA little deceptively named xhr . open ( 'POST' , '/path/to/\nendpoint' ); xhr . onload = function () { alert ( this . respon-\nseText ); }; var formData = new FormData (); formData .\nappend ( 'parameter' , value ); formData . append ( 'other-\nParameter' , value ); xhr . send ( formData );
```

Documentation: XMLHttpRequest

Also a bit more verbose than jQuery, but much simpler than you might've expected. Now here's the real kicker: It works in IE 7, and IE 5 with a little effort. IE actually pioneered XHR.

id="animations">Animations

This is where it starts to get more subjective and breaks backwards compatibility. Here's my opinion on the matter of transitions: dropping legacy browser support for fancy animations is acceptable. I don't think it's a problem if your website isn't pretty and animated on older browsers. Keep that in mind as we move on.

I want to animate the opacity of a .foobar when you hover over it. With jQuery:

```
class="highlight"> $( '.foobar' ). on ( 'mouseenter' , function\n() { $( this ). animate ( { opacity : 0.5 }, 2000 ); }). on ( 'mouse-\nleave' , function () { $( this ). animate ( { opacity : 1 }, 2000 ); });
```

Without jQuery, I wouldn't do this in Javascript. I'd use the magic of CSS animations:

Documentation: CSS animations

Much better, eh? Works in IE 10+. You can do much more complicated animations with CSS, but I can't think of a good demo, so that's an exercise left to the reader.

id="tree-traversal">Tree traversal

jQuery lets you navigate a tree pretty easily. Let's say you want to find the container of a button and remove all .foobar elements underneath it, upon clicking the button.

Nice and succinct. I'm sure you can tell the theme so far - the main advantage of jQuery is a less verbose syntax. Here's how it's done without jQuery:

```
class="highlight"> document . getElementById ( 'mybut-\nton' ). addEventListener ( 'click' , function () { var foobars =\nthis . parentElement . querySelectorAll ( '.foobar' ); for ( var\ni = 0 ; i
```

In conclusion

jQuery is, of course, based on JavaScript, and as a result, anything jQuery can do can be done without jQuery. Feel free to ask me if you're curious about how I'd do something else without jQuery.

I feel like adding jQuery is one of the first things a web developer does to their shiny new website. It just isn't really necessary in this day and age. That extra request, 91kb, and load time are probably negligible, but it's still a little less clean than it could be. There's no need to go back and rid all of your projects of jQuery, but I'd suggest that for your next one, you try to do without. Keep MDN open in the next tab over and I'm sure you'll get through it fine.

I don't think it's a problem if your website isn't pretty and animated on older browsers.

Drew DeVault

}

{

Please don't use Discord for FOSS projects

Drew DeVault

Six years ago, I wrote a post speaking out against the use of Slack for the instant messaging needs of FOSS projects. In retrospect, this article is not very good, and in the years since, another proprietary chat fad has stepped up to bat: Discord. It's time to revisit this discussion.

In short, using Discord for your free software/open source (FOSS) software project is a very bad idea. Free software matters — that's why you're writing it, after all. Using Discord partitions your community on either side of a walled garden, with one side that's willing to use the proprietary Discord client, and one side that isn't. It sets up users who are passionate about free software — i.e. your most passionate contributors or potential contributors — as second-class citizens.

By choosing Discord, you also lock out users with accessibility needs, for whom

the proprietary Discord client is often a nightmare to use. 1 Users who cannot

afford new enough hardware to make the resource-intensive client pleasant to use

are also left by the wayside. Choosing Discord is a choice that excludes poor

and disabled users from your community. Users of novel or unusual operating

systems or devices (i.e. innovators and early adopters) are also locked out of

the client until Discord sees fit to port it to their platform. Discord also

declines service to users in countries under US sanctions, such as Iran.

Privacy-conscious users will think twice before using Discord to participate in

your project, or will be denied outright if they rely on Tor or VPNs. All of

these groups are excluded from your community.

These problems are driven by a conflict of interest between you and Discord. Ownership over your chat logs, the right to set up useful bots, or to moderate your project's space according to your discretion; all of these are rights reserved by Discord and denied to you. The FOSS community, including users with accessibility needs or low-end computing devices, are unable to work together to innovate on the proprietary client, or to build improved clients which better suit their needs, because Discord insists on total control over the experience. Discord seeks to domesticate its users,

where FOSS treats users as peers and collaborators. These ideologies are fundamentally in conflict with one another.

You are making an investment when you choose to use one service over another.

When you choose Discord, you are legitimizing their platform and divesting from

FOSS platforms. Even if you think they have a bigger reach and a bigger

audience, 2 choosing them is a short-term, individualist play which signals a

lack of faith in and support for the long-term goals of the FOSS ecosystem as a

whole. The FOSS ecosystem needs your investment. FOSS platforms generally don't

have access to venture capital or large marketing budgets, and are less willing

to use dark patterns and predatory tactics to secure their market segment. They

need your support to succeed, and you need theirs. Why should someone choose to

use your FOSS project when you refused to choose theirs? Solidarity and mutual

support is the key to success.

There are great FOSS alternatives to Discord or Slack. SourceHut has been investing in IRC by building more accessible services like chat.sr.ht . Other great options include Matrix and Zulip . Please consider these services before you reach for their proprietary competitors.

Perceptive readers might have noticed that most of these arguments can be generalized. This article is much the same if we replace "Discord" with "GitHub", for instance, or "Twitter" or "YouTube". If your project depends on proprietary infrastructure, I want you to have a serious discussion with your collaborators about why. What do your choices mean for the long-term success of your project and the ecosystem in which it resides? Are you making smart investments, or just using tools which are popular or that you're already used to?

If you use GitHub, consider SourceHut 3 or Codeberg . If you use Twitter, consider Mastodon instead. If you use YouTube, try PeerTube . If you use Facebook... don't.

Your choices matter. Choose wisely.

id="fn:1">

Discord had to be sued to take this seriously. Updated at 2021-12-28 15:00 UTC: I asked a correspondent of mine who

}

{

Sourcehut's spartan approach to web design

Drew DeVault

Sourcehut is known for its brutalist design, with its mostly shades-of-gray appearance, conservative color usage, and minimal distractions throughout. This article aims to share some insights into the philosophy that guides this design, both for the curious reader and for the new contributor to the open-source project.

The most important principle is that `sr.ht` is an engineering tool first and foremost, and when you're there it's probably because you're in engineering mode. Therefore, it's important to bring the information you're there for to the forefront, and minimize distractions. In practice, this means that the first thing on any page to grab your attention should be the thing that brought you there. Consider the source file view on `git.sr.ht`. For reference, here are similar pages on GitHub and Gitlab.

The vast majority of the `git.sr.ht` page is dedicated to the source code we're

reading here, and it's also where most of the colors are. Your eye is drawn

straight to the content. Any additional information we show on this page is

directly relevant to the content: breadcrumbs to other parts of the tree, file

mode & size, links to other views on this repository. The nav can take you away

from this page, but it's colored a light grey to avoid being distracting and

each link is another engineering tool - no marketing material or fluff. Contrast

with GitHub: a large, dark, attention grabbing navbar with links to direct you

away from the content and towards marketing pages. If you're logged out, you get

a giant sign-up box which pushes the content halfway off the page. Colors here

are also distracting: note the large line of colorful avatars that catches your

eye despite almost certainly being unrelated to your purpose on this page.

Colors are used much more conservatively on sourcehut. If you log into

`builds.sr.ht` and visit the index page, you're greeted with a large blue "submit

manifest" button, and very little color besides. This is probably why you were

here - so it's made obvious and colorful so your eyes can quickly find it and

get on with your work. Other pages are similar: the `todo.sr.ht` tracker page has

a large form with a blue "submit" button for creating a new ticket, email views

on `lists.sr.ht` have a large blue "reply to thread" button, and

`man.sr.ht` has a large green button enticing new users

towards the tutorials. Red is also used throughout for dangerous actions, like

deleting things. Each button also is unambiguous and relies on the text within

itself rather than the text nearby: the `git.sr.ht` repository deletion page uses

"Delete \$reponame", rather than "Continue".

The last important point in sourcehut's design is the use of icons, or rather the lack thereof. Icons are used extremely conservatively on `sr.ht`. Interactive icons (things you are expected to click) are never shown without text that clarifies what happens when you click them. Informational icons usually have a tooltip which explains their meaning, and are quite rare - only used in cases where real estate limits the use of text. Assigning an icon to every action or detail is not necessary and would add more distractions to the screen.

I'm not a particularly skilled UI designer, so keeping it simple like this also helps to make a reasonably nice UI attainable for an engineer-oriented developer like me. Adding new pages is generally easy and requires little thought by applying these basic principles throughout, and the simple design doesn't take long to execute on. It's not perfect, but I like it and I've received positive feedback from my users.

}

{

Sharing Tinder's latest contributions to the open source community

Tinder · Tinder Tech

Authored By: Christopher Fuller

At first glance, Tinder might seem like a simple application. After all, Tinder is best known for its iconic Swipe Right feature, a simple gesture that revolutionized how people express interest and connect. But when you look under the hood, there are a lot of complexities to consider when building the experience that our users know and love today. People around the world indicate billions of Likes and Nopes per day, across a wide variety of iPhone models and screen sizes. When we roll out just one feature update, we have to understand how it'll impact every user on whatever device they're accessing the app from. I find this to be uniquely challenging, interesting and rewarding work.

As part of an initiative to contribute more to the software development community at large, I'm really excited to announce that we recently released our Layout code repository to the open source community. We are now publishing a few additional repositories as well.

We felt it was important to share what has worked well for our team in recent years. Both Layout and our mobile application architecture, Nodes , have long been essential parts of the Tinder iPhone app and its success to date. Before we explore our Layout repository in more detail, here's some context for how we got here.

How we got here

As Tinder grew in complexity, we repeatedly encountered challenges with reliability and performance in the iPhone app. To mitigate this, we focused our attention on extensibility, consistency and memory use. It quickly became clear that we needed to empower our engineers to continually add new features to the app safely without breaking existing functionality. The solution we devised, called the Nodes Architecture Framework, is a plugin-based approach which allows new app screens to be integrated into existing screens with only four lines of code.

A plugin validates the conditions that dictate whether a feature should be enabled. A single factory method call to "build" a feature is made possible using dependency injection code-generated at compile time. The logic of the feature is contained within a "context" object separate from the view to allow business rules to be tested in isolation. The context assumes the role of an interactor and responds to events, such as user actions. All output signals from a feature are received by the context object via delegation , uniquely referred to as a "listener" delegate. In other words, if screen A presents screen B, this means context A "listens" to context B. And a "flow" object assumes the role of a router responsible for starting other flows, as you can see in this example.

```
guard let builder = plugin.create() else { return } let flow = builder.build(withListener: con-
```

```
text) viewController.present(flow.getViewController(), animated: true) attach(starting: flow)
```

A side-effect of this approach is that plugins can be trivially added to plugin collections, known as "plugin lists", for feature versioning or A/B testing. The integration call-site does not need to know or care which version or variation of the feature is then built and started.

```
guard let builder = pluginList.create() ...
```

To optimize memory use, we introduced the concept of lifecycle to each feature. Dismissing a screen is an important lifecycle event that signifies that all memory for that feature must now be released. Debug builds can tap into this hook and warn when memory leaks occur.

```
await viewController.dismiss(flow.getViewController(), animated: true) detach(ending: flow)
```

We also developed custom Xcode templates to provide scaffolding when creating a new feature. This has fostered consistency across feature implementations to enable effortless context switching and ease of debugging common issues. The Nodes repository includes the generator that creates these Xcode templates.

The graph below is particularly interesting because it shows how adoption of the legacy architecture immediately flattened out the moment Nodes was released. As adoption of this new architecture was not mandated, this graph demonstrates that our teams immediately found Nodes to be beneficial and saw value in moving away from the legacy architecture.

— Nodes adoption since its introduction at the beginning of 2020 as compared to legacy architecture.

Why Layout was critical to our success

While the Nodes framework provides essential structure for our mobile engineers to architect their features, one of the most important aspects they focus on next is how the screens of the app look.

Since the Tinder iPhone app has been primarily built with UIKit , and we wanted to make it as easy as possible for our engineers to define UI layouts, we developed a new high fidelity DSL syntax, called Layout , to easily visualize the screens the code represents.

In the evolving mobile development landscape, declarative UI frameworks such as SwiftUI and Jetpack Compose offer benefits such as significantly reduced learning curves. While not a declarative framework, Layout utilizes a similar declarative style, in this case for the UIKit framework.

We've used Layout, a wrapper around Apple's Auto Layout SDK, as part of our iPhone codebase since 2017 to programmatically define the screens of the app.

There are several key advantages to Layout that have

}

{

More speculations on arenas in C++

Chris Wellons (null program)

Update October 2025: further enhancements .

Patrice Roy's new book, *C++ Memory Management* , has made me more conscious of object lifetimes. C++ is stricter than C about lifetimes, and common, textbook memory management that's sound in C is less so in C++ — more than I realized . The book also presents a form of arena allocation so watered down as to enjoy none of the benefits. (Despite its precision otherwise, the second half is also littered with integer overflows lacking the appropriate checks , and near the end has some pointer overflows invalidating the check.) However, I'm grateful for the new insights, and it's made me revisit my own C++ arena allocation . In this new light I see I got it subtly wrong myself!

Surprising to most C++ programmers, but not language lawyers, idiomatic C memory allocation was ill-formed in C++ until recently :

```
class="highlight">int * newint ( int v ) { int * r = ( int * )
malloc ( sizeof ( * r )); if ( r ) { * r = v ; //
```

```
class="highlight">int * newint ( int v ) { void * r = malloc
( sizeof ( int )); if ( r ) { return new ( r ) int { v }; } return
nullptr ; }
```

That's no good as a C/C++ polyglot, though per the differing old semantics that was impossible anyway without macros. Which is basically cheating. An important detail: The corrected version has no casts, and it returns the result of `new` . That's important because only the pointer returned by `new` is imbued as a pointer to the new lifetime, not `r` . There are no side effects affecting the provenance of `r` , which still points to raw memory as far as the language is concerned.

With that in mind let's revisit my arena from last time, which does not necessarily benefit from the recent changes, not being one of the special case C standard library functions:

```
template T * alloc ( Arena * a , ptrdiff_t count = 1 ) { ptrdiff_t
size = sizeof ( T ); ptrdiff_t pad = - ( uintptr_t ) a -> beg &
( alignof ( T ) - 1 ); assert ( count end - a -> beg - pad ) / size ); /
\ OOM policy T * r = ( T * )( a -> beg + pad ); a -> beg += pad
+ count * size ; for ( ptrdiff_t i = 0 ; i
```

```
class="highlight">template T * alloc ( Arena * a , ptrdiff_t
count = 1 ) { ptrdiff_t size = sizeof ( T ); ptrdiff_t pad = -
( uintptr_t ) a -> beg & ( alignof ( T ) - 1 ); assert ( count end - a
-> beg - pad ) / size ); /\ OOM policy void * r = a -> beg + pad ;
a -> beg += pad + count * size ; return new ( r ) T [ count ]{} ; }
```

Wow... that's actually much better anyway. No explicit casts, no loop. Why didn't I think of this in the first place? The catch is I can't forward constructor arguments, `emplace`-style — the part that gave me the trouble with perfect forwarding — but that's for the best. Forwarding more than once was unsound, made more obvious by `new[]` .

Caveat: This only works starting in C++20, and strictly with operator `new[](size_t, void *)` . Any other placement `new[]` may require array overhead — e.g. it prepends an array size so that `delete[]` can run non-trivial destructors — which is unknowable and therefore impossible to provide or align correctly. Overhead for placement `new[]` is nonsense, of course, but as of this writing, all three major C++ compilers do it and essentially have broken custom placement `new[]` .

Since I'm thinking about lifetimes, what about the other end? My arena does not call destructors, by design, and starts new lifetimes on top of objects that are technically still alive. Is that undefined behavior? As far as I can tell this is allowed , even for non-trivial destructors, with the caveat that it might leak resources. In this case the resource is memory managed by the arena, so that's fine of course.

So addressing pointer provenance also produced a nicer definition. What a great result from reading that book! While researching, I noticed Jonathan Müller, who personally gave me great advice and feedback on my previous article, talked about lifetimes just a couple weeks later. I recommend both.

}

{

It's not okay to pretend your software is open source

Drew DeVault

Unfortunately, I find myself writing about the Commons Clause again. For those not in the know, the Commons Clause is an addendum designed to be added to free software licenses. The restrictions it imposes (you cannot sell the software) makes the resulting franken-license nonfree. I'm not going to link to the project which brought this subject back into the discussion - they don't deserve the referral - but the continued proliferation of software using the Commons Clause gives me reason to speak out against it some more.

One of my largest complaints with the Commons Clause is that it hijacks language used by open source projects to proliferate nonfree software, and encourages software using it to do the same. Instead of being a new software license, it tries to stick itself onto other respected licences - often the Apache 2.0 license. The name, "Commons Clause", is also disingenuous, hijacking language used by respected entities like Creative Commons. In truth, the Commons Clause serves to remove software from the commons¹. Combining these problems gives you language like "Apache+Commons Clause", which is easily confused with [Apache Commons][apache-commons].

Projects using the Commons Clause have also been known to describe their license as "permissive" or "open", some even calling their software "open source". This is dishonest. FOSS refers to "free and open source software". The former, free software, is defined by the free software definition, published by GNU. The latter, open source software, is defined by the open source definition, published by the OSI. Their definitions are very similar, and nearly all FOSS licenses qualify under both definitions. These are unambiguous, basic criteria which protects developers, contributors, and users of free and open source software. These definitions are so basic, important and well-respected that dismissing them is akin to dismissing climate change.

Claiming your software is open source, permissively licensed, free software, etc, when you use the Commons Clause, is lying. These lies are pervasive among users of the Commons Clause. The page listing Redis Modules, for example, states that only software under an OSI-approved license is listed. Six of the modules there are using nonfree licenses, and antirez seems content to ignore the problem until we forget about it. They're in for a long wait - we're not going to forget about shady, dishonest, and unethical companies like Redis Labs.

I don't use nonfree software², but I'm not going to sit here and tell you not to make nonfree software. You have every right to license your work in any way you choose. However, if you choose not to use a FOSS license, you need to own up to it. Don't pretend that your software is something it's not. There are many benefits to being a member of the free software community, but you are not entitled to them if your software isn't. This behavior has to stop.

Finally, I have some praise to offer. Dgraph was briefly licensed under Apache plus the Commons Clause, and had the sort of misleading and false information this article decries on their marketing website, docs, and so on. However, they've rolled it back, and Dgraph is now using the Apache 2.0 license with no modifications. Thank you!

id="fn:1">

This is why I often refer to it as the "Anti-Commons Clause", though I felt that was a bit too Stallman-esque for this article. [apache-commons]: <http://commons.apache.org/> ↩

id="fn:2">

Free as in freedom, not as in free beer. ↩

}

{

Spooky action at a distance

Drew DeVault

Einstein famously characterized the strangeness of quantum mechanics as “spooky action at a distance”, which, if I had to pick one phrase about physics to be my favorite, would be a strong contender. I like to relate this to programming language design: there are some language features which are similarly spooky. Perhaps the most infamous of these is operator overloading. Consider the following:

```
x + y
```

If this were written in C, without knowing anything other than the fact that this code compiles correctly, I can tell you that `x` and `y` are numeric types, and the result is their sum. I can even make an educated guess about the CPU instructions which will be generated to perform this task. However, if this were a language with operator overloading... who knows? What if `x` and `y` are some kind of some Vector class? It could compile to this:

```
Vector::operator_plus(x, y)
```

The performance characteristics, consequences for debugging, and places to look for bugs are considerably different than the code would suggest on the surface. This function call is the “spooky action” — and the distance between the “+” operator and the definition of its behavior is the “distance”.

Also consider if `x` and `y` are strings: maybe “+” means concatenation? Concatenation often means allocation, which is a pretty important side-effect to consider. Are you going to thrash the garbage collector by doing this? Is there a garbage collector, or is this going to leak? Again, using C as an example, this case would be explicit:

```
char *new = malloc(strlen(x) + strlen(y) + 1); strcpy(new, x);
strcat(new, y);
```

If the filename of the last file you had open in your text editor ended in `.rs`, you might be frothing at the mouth after reading this code. Strictly for the purpose of illustrating my point, however, consider that everything which happens here is explicit, opt-in to the writer, and obvious to the reader.

That said, C doesn’t get off scott-free in this article. Consider the following code:

```
int x = 10, y = 20; int z = add(x, y); printf(“%d + %d = %d\n”,
x, y, z);
```

You may expect this to print out `10 + 20 = 30`, and you would be forgiven for your naivety.

```
$ cc -o test test.c $ ./test 30 + 20 = 30
```

The savvy reader may have already figured out the catch: `add` is not a function.

```
#define add(x, y) x += y
```

The spooky action is the mutation of `x`, and the distance is between the apparent “callsite” and the macro definition. This is spooky because it betrays the reader’s expectations: it looks and smells like a function call, but it does something which breaks the contract of function calls. Some languages do this better, by giving macros an explicit syntax like `name!(args...)`, but, personally, I still don’t like it.

Language features like this are, like all others, a trade-off. But I’m of the opinion that this trade is unwise: you’re wagering readability, predictability, debuggability, and more. These features are toxic to anyone seeking stable, robust code. They certainly have no place in systems programming.

Strictly for the purpose of illustrating my point, however, consider that everything which happens here is explicit, opt-in to the writer, and obvious to the reader.

Drew DeVault

}

{

China

Drew DeVault

This article will be difficult to read and was difficult to write. I hope that you can stomach the uncomfortable nature of this topic and read my thoughts in earnest. I usually focus on technology-related content, but at the end of the day, this is my personal blog and I feel that it would betray my personal principles to remain silent. I've made an effort to provide citations for all of my assertions.

Note: if you are interested in conducting an independent review of the factuality of the claims expressed in this article, please contact me .

The keyboard I'm typing these words into bears "Made in China" on the bottom. The same is true of the monitor I'm using to edit the article. It's not true of all of my electronics — the graphics processing unit which is driving the monitor was made in Taiwan 1 and my phone was made in Vietnam. 2 Regardless, there's no doubt that my life would be, to some degree, worse off if not for trade with China. Despite this, I am prepared to accept the consequences of severing economic relations with China.

How bad would being cut-off from China's economy be? We're a net importer from China, and by over 4 times the volume. 3 Let's assume, in the worst case, trade ties were completely severed. The United States would be unable to buy \$155B worth of electronics, which we already have domestic manufacturing capabilities for 4 and which have a productive life of several years. We could definitely stand to get used to repairing and reusing these instead of throwing them out. We'd lose \$34B in mattresses and furniture — same story. The bulk of our imports from China are luxury goods that we can already make here at home 5 — it's just cheaper to buy them from China. But cheaper for whom?

This gets at the heart of the reason why we're tied to China economically. It's economically productive for the 1% to maintain a trade relationship with China. The financial incentives don't help any Americans, and in fact, most of us are hurt by this relationship. 6 Trade is what keeps us shackled to the Chinese Communist Party government, but it's not beneficial to anyone but those who are already obscenely rich, and certainly not for our poorest — who, going into 2020, are as likely to be high school dropouts as they are to be doctors. 7

So, we can cut off China. Why should we? Let's lay out the facts: China is conducting human rights violations on the largest scale the world has seen since Nazi Germany. China executes political prisoners 8 and harvests their organs for transplant to sick elites on an industrial scale, targeting and killing civilians based on not only political, but also ethnic and religious factors. This is commonly known as genocide. China denies using the organs of prisoners, but there's credible doubt 9 from the scientific community.

Recent evidence directly connecting executions to organ harvesting is somewhat unreliable, but I don't think China deserves the benefit of the doubt. China is a world leader in executions, and is believed to conduct more executions than the rest of the world combined. 10 Wait times for organ transplantation are extraordinarily low in China, 11 on the order of weeks — in most of the developed world these timeframes are measured in terms of years, 12 and China has been unable to explain the source for tens of thousands of transplants in the past 13 . And, looking past recent evidence, China directly admitted to using the organs of executed prisoners in 2005. 14

These atrocities are being committed against cultural minorities to further China's power. The UN published a statement in August 2018 stating that they have credible reports of over a million ethnic Uighurs being held in internment camps in Xinjiang, 15 imprisoned with various other ethnic minorities from the region. Leaks in November 2019 reported by the New York Times showed that China admits the imprisoned have committed no crimes other than dissent, 16 and that the camps were to be run with, quote, "absolutely no mercy".

It's nice to believe that we would have stood up to Nazi Germany if we had been there in the 1940's. China is our generation's chance to prove ourselves of that conviction. We talk a big game about fighting against white nationalists in our own country, and pride ourselves on standing up against "fascists". It's time we turned attention to the real fascists, on the world stage.

Instead, the staunch capitalism of America, and the West as a whole, has swooped in to leverage Chinese fascism for a profit. Marriott Hotels apologized for listing Hong Kong, Macau, and Taiwan as countries separate from China. 17 Apple removed the Taiwanese flag from iOS in China and the territories it claims. 18 Activision/Blizzard banned several players for making pro-Hong Kong statements in tournaments and online. 19 These behaviors make me ashamed to be an American.

Fuck that.

A brief history lesson: Hong Kong was originally controlled by the United Kingdom at the end of the Opium Wars. It's beyond the scope of this article, but it'll suffice to say that the United Kingdom was brutal and out of line, and the end result is that Hong Kong became a British colony. Because of this, it was protected from direct Chinese influence during China's turbulent years following, and they were insulated from the effects of the Great Leap Forward and the Cultural Revolution, which together claimed tens of millions of lives and secured the Communist Party of China's power into the present.

On July 1st, 1997, the Sino-British Joint Declaration went into effect , and Hong Kong was turned over to China. The agreement stipulated that Hong Kong would remain effectively autonomous and self-governing for a period of 50 years — until 2047. China has been gradually and

}

{

Using the right tool for the job

Drew DeVault

One of the most important choices you'll make for the software you write is what you write it in, what frameworks you use, the design methodologies to subscribe to, and so on. This choice doesn't seem to get the respect it's due. These are some of the only choices you'll make that you cannot change. Or, at least, these choices are among the most difficult ones to change.

People often question why TrueCraft is written in C# next to projects like Sway in C, alongside KnightOS in Assembly or sr.ht in Python. It would certainly be easier from the outset if I made every project in a language I'm comfortable with, using tools and libraries I'm comfortable with, and there's certainly something to be had for that. That's far from being the only concern, though.

A new project is a great means of learning a new language or framework - the only effective means, in fact. However, the inspiration and drive for new projects doesn't come often. I think that the opportunity for learning is more important than the short term results of producing working code more quickly. Making a choice that's more well suited to the problem at the expense of comfort will also help your codebase in the long run. Why squander the opportunity to choose something unfamiliar when you have the rare opportunity to start working on a new project?

I'm not advocating for you to use something new for every project, though. I'm suggesting that you detach your familiarity with your tools from the decision-making process. I often reach for old tools when starting a new project, but I have learned enough about new tools that I can judge what projects are a good use-case for them. Sometimes this doesn't work out, too - I just threw away and rewrote a prototype in C after deciding that it wasn't a good candidate for Rust.

Often it does work out, though. I'm glad I chose to learn Python for MediaCrush despite having no experience with it (thanks again for the help with that, Jose!). Today I still know it was the correct choice and knowing it has hugely expanded my programming skills, and without that choice there probably wouldn't have been a Kerbal Stuff or a sr.ht or likely even the new API we're working on at Linode. I'm glad I chose to learn C for z80e, though I had previously written emulators in C#. Without it there wouldn't be many other great tools in the KnightOS ecosystem written in C, and there wouldn't be a Sway or an aerc. I'm glad I learned ES6 and React instead of falling back on the familiar Knockout.js when building prototypes for the new Linode manager as well.

Today, I have a mental model of the benefits and drawbacks of a lot of languages, frameworks, libraries, and platforms I don't know how to use. I'm sort of waiting for projects that would be well suited to things like Rust or Django or Lisp or even Plan 9. Remember, the skills you already know

make for a great hammer, but you shouldn't nail screws to the wall.

}

{

Speculations on arenas and non-trivial destructors

Chris Wellons (null program)

As I continue to reflect on arenas and lifetimes in C++, I realized

that dealing with destructors is not so onerous. In fact, it does not even

impact my established arena usage ! That is, implicit RAII-style

deallocation at scope termination, which works even in plain old C. With a

small change we can safely place resource-managing objects in arenas, such

as those owning file handles, sockets, threads, etc. (Though the ideal

remains resource management avoidance when possible.) We can also

place traditional, memory-managing C++ objects in arenas, too. Their own

allocations won't come from the arena — either because they lack the

interfaces to do so, or they're simply ineffective at it (pmr) —

but they will reliably clean up after themselves. It's all exception-safe,

too. In this article I'll update my arena allocator with this new feature.

The change requires one additional arena pointer member, a bit of overhead

for objects with non-trivial destructors, and no impact for other objects.

I continue to title this “speculations” because, unlike arenas in C, I have not (yet?) put these C++ techniques into practice in real software. I haven't refined them through use. Even ignoring its standard library as I do here, C++ is an enormously complex programming language — far more so than C — and I'm less confident that I'm not breaking a rule by accident. I only want to break rules with intention!

As a reminder here's where we left things off:

```
template T * raw_alloc ( Arena * a , ptrdiff_t count = 1 )
{ ptrdiff_t size = sizeof ( T ); ptrdiff_t pad = - ( uintptr_t ) a -
> beg & ( alignof ( T ) - 1 ); if ( count >= ( a -> end - a -> beg -
pad ) / size ) { throw std :: bad_alloc {}; } /\ OOM policy } void
* r = a -> beg + pad ; a -> beg += pad + count * size ; return
new ( r ) T [ count ];; }
```

I used throw when out of memory mainly to emphasize that this works, but you're free to pick whatever is appropriate for your program. Remember, that's the entire allocator, including implicit deallocation, sufficient to fulfill the allocation needs for most programs, though they must be designed for it. Also note that it's now raw_alloc , as we'll be writing a new, enhanced alloc that builds upon this one.

Also a reminder on usage, I'll draw on an old example, updated for C++:

```
class="highlight"> wchar_t * towidechar ( Str , Arena * ); /\
/\ convert to UTF-16 Str slurpfile ( wchar_t * path ); /\ read
an entire file Slice split ( Str , char , Arena * ); /\ split on
delimiter
```

```
Slice getlines ( Str path , Arena * perm , Arena scratch )
{ /\ Use scratch for path conversion, auto-free on return
wchar_t * wpath = towidechar ( path , & scratch );
```

```
/\ Use perm for file contents, which are returned Str buf =
slurpfile ( wpath , perm );
```

```
/\ Use perm for the slice, pointing into buf return split ( buf ,
'n' , perm ); }
```

Changes to scratch do not persist after getlines returns, so objects allocated from that arena are automatically freed on return. So far this doesn't rely on C++ RAII features, just simple value semantics. It works well because all the objects in question have trivial destructors. But suppose there's a resource to manage:

```
class="highlight"> struct TcpSocket { int socket = :: socket
( AF_INET , SOCK_STREAM , 0 ); TcpSocket () = default ; Tcp-
Socket ( TcpSocket & ) = delete ; void operator = ( TcpSocket
& ) = delete ; /\ TODO: move ctor/operator  TcpSocket ()
{ if ( socket >= 0 ) close ( socket ); } operator int () { return
socket ; } ; }
```

If we allocate a TcpSocket in an arena, including as a member of another object, the destructor will never run unless we call it manually. To deal with this we'll need to keep track of objects requiring destruction, which we'll do with a linked list of destructors, forming a LIFO stack:

Each Dtor points to a homogeneous array, a count (typically one), and a pointer to a function that knows how to destroy these objects. The linked list itself is heterogeneous, with dynamic type. The function pointer is like a kind of type tag. The dtor functions will be generated using a template function:

```
class="highlight"> template void destroy ( void * ptr ,
ptrdiff_t count ) { T * objects = ( T * ) ptr ; for ( ptrdiff_t i =
count - 1 ; i >= 0 ; i - ) { objects [ i ]. ~ T (); } }
```

Notice it destroys end-to-beginning, in reverse order that these objects would be instantiated by placement new[] . It's essentially a placement delete[] . An arena initializes with an empty list of Dtors as a new member:

}

{

Process scheduling and multitasking in KnightOS

Drew DeVault

I'm going to do some blogging about technical decisions made with KnightOS . It's an open-source project I've been working on for the past four years to build an open-source Unix-like kernel for TI calculators (in assembly). It's been a cool platform on top of which I can research low level systems concepts and I thought I'd share some of my findings with the world.

So, first of all, what is scheduling? For those who are completely out of the

loop, I'll explain what exactly it is and why it's necessary. Computers run on

a CPU, which executes a series of instructions in order. Each core is not

capable of running several instructions concurrently. However, you can run

hundreds of processes at once on your computer (and you probably are doing so

as you read this article). There are a number of ways of accomplishing, but the

one that suits the most situations is preemptive multitasking . This is what

KnightOS uses. You see, a CPU can only execute one instruction after another,

but you can "raise an interrupt". This will halt execution and move to some

other bit of code for a moment. This can be used to handle various events (for

example, the GameBoy raises an interrupt when a button is pressed). One of

these events is often a timer, which raises an interrupt at a fixed interval.

This is the mechanism by which preemptive multitasking is accomplished.

Let's say for a moment that you have two programs loaded into memory and running, at addresses 0x1000 and 0x2000. Your kernel has an interrupt handler at 0x100. So if program A is running and an interrupt fires, the following happens:

0x1000 is pushed to the stack as the return address

The program counter is set to 0x100 and the interrupt runs

The interrupt concludes and returns, which pops 0x1000 from the stack and into the program counter.

Once the interrupt handler runs, however, the kernel has a chance to be sneaky:

0x1000 is pushed to the stack as the return address

The program counter is set to 0x100 and the interrupt runs

The interrupt removes 0x1000 from the stack and puts 0x2000 there instead

The interrupt concludes and returns, which pops 0x2000 from the stack and into the program counter.

Now the interrupt has switched the CPU from program A to program B. And the next time an interrupt occurs, the kernel can switch from program B to program A. This event is called a "context switch". This is the basis of preemptive multitasking. On top of this, however, there are lots of ideas around which processes should get CPU time and when. Some systems have more complex schedulers, but KnightOS runs on limited hardware and I wanted the context switch to be short and sweet so that the running processes get as much of the CPU as possible. I'll explain the simple KnightOS scheduling algorithm here. First, its goals:

Short and simple context switches

Ability to suspend processes when not in foreground

Ability to run background processes

What KnightOS uses is a simple round robin with the ability to suspend threads. That is, we have a list of processes and then some flags, among which is whether or not the processes is currently suspended. So say we have this list of processes in memory:

1: PC=0x2000, not suspended

2: PC=0x2000, not suspended

3: PC=0x2000, suspended

4: PC=0x2000, not suspended

As process 1 is running and an interrupt fires, the kernel looks at this table and picks the next non-suspended process to run - process 2. On the next interrupt, it does it again, skipping process 3 and giving time to process 4.

To actually implement this, we have to think about the stack. KnightOS runs on z80 processors, which have a single stack and a shared memory space. The CPU uses the PC register to keep track of which address the current instruction is at. That is, say you compile this code:

ld a, 10
inc a
ld (hl), a

This compiles to the machine code 3E 0A 3C 77. Say we load this program at 0x8000 - then 0x8000 will point to ld a, 10 . When the CPU finishes executing this instruction, it advances PC to 0x8002 (since ld a, 10 is a two-byte instruc-

}

standard-article(title: [Updating to the Latest Ansible on Ubuntu], author: [Scott Johnson (fuzzyblog)], source-name: [Scott Johnson (fuzzyblog)], images: (), paragraphs: ([One of the tricky bits about Ansible is that new features in the “language” always require the latest version of Ansible itself. And, while logical, this can easily bite you. Here’s an example:], [class=“highlight”> TASK [deploy_hate-language-modeling_systemd_start : just force systemd to re-execute itself (2.8 and above)] *** fatal : [aws_master2]: FAILED! => {“changed” : false , “msg” : “Unsupported parameters for (systemd) module: daemon_reexec Supported parameters include: daemon_reload, enabled, masked, name, no_block, state, user” } to retry, use : -limit @/home/ubuntu/ansible/playbook_deploy_hate-language-modeling_master.retry], [What that translates to is that Ansible itself doesn’t know how to use the parameter daemon_reexec . And when you’ve been using an Ansible feature successfully and then it doesn’t work on the server, that usually is a rock solid signal “update ansible”], [Here’s how to do this on Ubuntu:],), insert-map: (:), word-count: 155, edited-for-length: false, debug-mode: false,)

standard-article(title: [A Week of Whole 30 Meals From My Recipes], author: [Scott Johnson (fuzzyblog)], source-name: [Scott Johnson (fuzzyblog)], images: (), paragraphs: ([My eldest son needs to lose about 15 pounds before he leaves for boot camp in the Marines. So here is a week of Whole 30 Meals planned out for him. Here are more details on Whole 30 (scroll down about 1/2 way)]., [id=“monday”>Monday], [Breakfast: 3 Squishy Eggs , Black Coffee], [Lunch: Hard boiled eggs, nuts, fruit], [Dinner: Grilled Chicken Breast, Baked potato with Olive Oil (no dairy), Salad with homemade dressing], [Snack: handful of cashews], [Sweet: Lara bar], [Reward: Dates], [Beverage: LaCroix Water], [id=“tuesday”>Tuesday], [Breakfast: 2 Fried Eggs], [Lunch: Hard boiled eggs, nuts, fruit], [Dinner: Garlicky Pork Chops, Roasted Carrots , Salad], [Snack: Mixed Nuts; no Peanuts], [Sweet: Lara bar], [Reward: Dates], [Beverage: LaCroix Water], [id=“wednesday”>Wednesday], [Breakfast: Scrambled Eggs], [Lunch: Burger patty, nuts, fruit], [Dinner: Steak, Broccoli, Salad], [Snack: Terra Chips], [Sweet: Lara bar], [Reward: Apple], [Beverage: LaCroix Water], [id=“thursday”>Thursday], [Breakfast: Omelet], [Lunch: Hard boiled eggs, nuts, fruit], [Dinner: Burger, Hash Browns], [Snack: Cashews, Mixed Nuts], [Sweet: Lara Bar], [Reward: Berries], [Beverage: LaCroix Water], [id=“friday”>Friday], [Breakfast: Scrambled Eggs], [Lunch: Hard boiled eggs, nuts, fruit], [Dinner: Steak, Left Over Hash Browns, Sautéed Bell Peppers and Onions], [Snack: Lara Bar], [Sweet: Berries], [Reward: Dates], [Beverage: LaCroix Water], [id=“saturday”>Saturday], [Breakfast: Omelet], [Lunch: Burgers Wrapped in Lettuce], [Dinner: Left Over Steak, Mashed Potatoes Made with Chicken Broth], [Snack: Cashews], [Sweet: Lara Bar], [Reward: Berries], [Beverage: LaCroix Water], [id=“sunday”>Sunday], [Breakfast: Fried Eggs], [Lunch: Hard Boiled Eggs, Salad], [Dinner: Meat Loaf, Mashed Potatoes], [Snack: Lara Bar], [Sweet: Lara Bar], [Reward: Berries], [Beverage: LaCroix Water],), insert-map: (:), word-count: 250, edited-for-length: false, debug-mode: false,)

brief-group(([

The Hacker News, The Hacker News (cybersecurity): Some weeks in security feel loud. This one feels sneaky. Less big dramatic fireworks, more of that slow creeping sense that too many people are getting way too comfortable abusing things they probably shouldn’t even be touching. There’s a little bit of everything in this one, too. Weird delivery tricks, old problems coming back in slightly worse forms, shady infrastructure doing

], [

Scott Johnson (fuzzyblog), Scott Johnson (fuzzyblog): class=“center”>

Your Rails 6 master key is located in:

class=“language-plaintext highlighter-rouge”>

class=“highlight”> config/credentials/production.key

and it is likely stored as a hidden file. I believe that this is generated when the project is generated but I’m not 100% certain.

See this Reddit post .

], [

Axel Rauschmayer (2ality), Axel Rauschmayer (2ality): TypeDoc now lets us refer to parts of other files via {@includeCode} . In this blog post, I explain how that works and why it’s useful.

], [

The Hacker News, The Hacker News (cybersecurity): Threat actors with ties to Iran successfully broke into the personal email account of Kash Patel, the director of the U. S. Federal Bureau of Investigation (FBI), and leaked a cache of photos and other documents to the internet. Handala Hack Team, which carried out the breach, said on its website that Patel “will now find his name among the list of successfully hacked victims.” In a statement

], [

Christophe Yammouni, Drivy / Getaround Tech: External third-party-services APIs are useful: they allow you to benefit from the expertise and knowledge that others have acquired on a specific subject - a subject which is not your area of expertise and not the problem in hand. It would take too much time and effort to build and maintain such a service yourself.

], [

The Hacker News, The Hacker News (cybersecurity): The U. S. Cybersecurity and Infrastructure Security Agency (CISA) on

Friday added a critical security flaw impacting F5 BIG-IP Access Policy Manager (APM) to its Known Exploited Vulnerabilities (KEV) catalog, citing evidence of active exploitation. The vulnerability in question is CVE-2025-53521 (CVSS v4 score: 9.3), which could allow a threat actor to achieve remote code execution. “When a

],))

The New Mirror

Vol. 1, No. 011 · 2026-03-30

Curated and composed automatically.